

RepairAgent: An Autonomous, LLM-Based Agent for Program Repair

ISLEM BOUZENIA, University of Stuttgart, Germany

PREMKUMAR DEVANBU, UC Davis, USA

MICHAEL PRADEL, University of Stuttgart, Germany

Automated program repair has emerged as a powerful technique to mitigate the impact of software bugs on system reliability and user experience. This paper introduces RepairAgent, the first work to address the program repair challenge through an autonomous agent based on a large language model (LLM). Unlike existing deep learning-based approaches, which prompt a model with a fixed prompt or in a fixed feedback loop, our work treats the LLM as an agent capable of autonomously planning and executing actions to fix bugs by invoking suitable tools. Repair Agent freely interleaves gathering information about the bug, gathering repair ingredients, and validating fixes, while deciding which tools to invoke based on the gathered information and feedback from previous fix attempts. Key contributions that enable RepairAgent include a set of tools that are useful for program repair, a dynamically updated prompt format that allows the LLM to interact with these tools, and a finite state machine that guides the agent in invoking the tools. Our evaluation on the popular Defects4J dataset demonstrates RepairAgent’s effectiveness in autonomously repairing 164 bugs, including 39 bugs not fixed by prior techniques. Interacting with the LLM imposes an average cost of 270k tokens per bug, which, under the current pricing of OpenAI’s GPT-3.5 model, translates to 14 cents per bug. To the best of our knowledge, this work is the first to present an autonomous, LLM-based agent for program repair, paving the way for future agent-based techniques in software engineering.

ACM Reference Format:

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2026. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. 1, 1 (July 2026), 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Software bugs can lead to system failures, security vulnerabilities, and compromised user experience. Manually fixing bugs demands significant time and user effort, but **Automated Program Repair (APR)** can significantly reduce the effort via automatic bug resolution. Various approaches have been explored in this regard [Le Goues et al. 2019], incl. techniques based on manually designed [Le Goues et al. 2012], [Liu et al. 2019] and (semi-)automatically extracted [Kim et al. 2013], [Bader et al. 2019] fix patterns, based on symbolic constraints [Nguyen et al. 2013], [Xuan et al. 2016], and various machine learning based approaches [Long and Rinard 2016], [Gupta et al. 2017], [Tufano et al. 2019].

The current state-of-the-art in APR is predominated by LLMs. In the first generation of LLM based APR, the model expects a prompt with the buggy code and produces the fixed version [Xia et al. 2023], [Jiang et al. 2023], but further development introduces an iterative approach: the LLM is queried repeatedly based on feedback obtained from previous fix attempts [Xia and Zhang 2023], [Kang et al. 2023], [Ye and Monperrus 2024].

But this approach has a few limitations since the hard-coded feedback loops in the LLMs do not allow them to gather information about existing bugs or existing code that may provide them with debugging ingredients, unlike manual

Authors’ Contact Information: Islem Bouzenia, fi_bouzenia@esi.dz, University of Stuttgart, Germany; Premkumar Devanbu, ptdevanbu@ucdavis.edu, UC Davis, USA; Michael Pradel, michael@binaervarianz.de, University of Stuttgart, Germany.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

human debugging which typically incorporates a temporal interleaving of gathering information to gain understanding of the bug, searching code that might help in debugging and then attempting various versions of fixes [Ko et al. 2006], [Böhme et al. 2017].

This paper represents **RepairAgent**, the first autonomous, LLM based agent for automated program repair. The LLM is treated as an autonomous agent capable of planning and executing actions to fix a bug. The LLM is equipped with a set of repair-specific tools. How and when to use these tools aren't hard coded, rather the LLM gets to autonomously decide.

The approach is enabled via three key components: a general purpose LLM e.g. GPT-3.5, a set of tools for the LLM to use (in this approach, 14 tools in total have been used), and a middleware to establish communication between the tools and the LLM.

The novelty of the approach was tested using all 835 bugs of the Defects4J [Just et al. 2014] dataset. 164 bugs were successfully fixed, including 39 bugs that were not fixed by prior state-of-the-art work [Xia and Zhang 2023] [Ye and Monperrus 2024].

The interaction with the LLM imposes a cost of 270K tokens per bug on average, which translates to about **14 cents** per bug.

In summary, this paper contributes the following:

- An autonomous, LLM-based agent for program repair.
- A dynamically updated prompt format that guides the LLM through the bug repair process.
- A set of tools that enable a LLM to perform steps a human developer would take when fixing a bug.
- A middleware that guides via a finite state machine how the LLM interacts with the tools.
- Empirical evidence that RepairAgent establishes a new state of the art in program repair.
- We release our implementation as open-source: <https://github.com/sola-st/RepairAgent>.

To the best of our knowledge, this is the first work to present an autonomous, LLM-based agent for program repair, paving the way for future agent-based techniques in software engineering.

2 Background (LLM-Based Autonomous Agents)

Because of extensive training, LLMs have demonstrated remarkable abilities in achieving human-level performance in a variety of tasks [Bubeck et al. 2023]. A promising way to use these LLMs are LLM-based agents, which mean LLM-based techniques with two properties: (1) The LLM can autonomously plan for and execute a set of actions to achieve a particular goal, unlike hard-coded algorithmic processes, and (2) The actions by the LLM can invoke external tools, enabling it to interact with its environment [Schick et al. 2023] [Patil et al. 2023]. In the context of Software Engineering, the tools could be tools usually used by developers, e.g., as part of an integrated development environment (IDE). Basically, we query the LLM with the current state of the world, the goal and a set of actions to perform. The feedback is integrated into the next prompt.

3 Method

3.1 Overview

Fig. 1 provides an overview of RepairAgent.

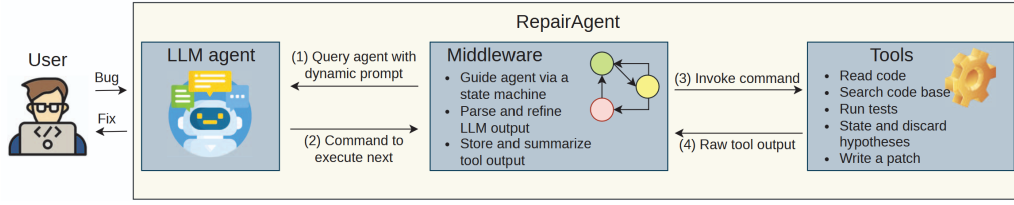


Fig. 1. Overview of RepairAgent.

3.2 Terminology

RepairAgent proceeds in multiple cycles:

Definition 1 (Cycle). A *cycle* represents one round of interaction with the LLM agent, which consists of the following steps:

- (1) Query the agent
- (2) Post-process the response
- (3) Execute the command suggested by the agent
- (4) Update the dynamic prompt based on the command's output

In each cycle, the approach queries the LLM once. The input to the model is updated based on commands invoked by the LLM, and their results, in previous cycles. We call the model input a dynamic prompt:

Definition 2 (Dynamic prompt). The *dynamic prompt* is a sequence of text sections $P = [s_0, s_1, \dots, s_n]$, where each section s_i is one of the following (where $s_i(c)$ refers to a section during a cycle c):

- A *static section*, which remains the same across all cycles, i.e., $s_i(c) = s_i(c')$ for all c, c' .
- A *dynamic section*, which may differ across cycles, i.e., there may exist c, c' with $s_i(c) \neq s_i(c')$.

3.3 Dynamic Prompting of the RepairAgent

As listed in Table 1, the prompt contains multiple static and dynamic sections.

Table 1. Sections of the dynamically updated prompt within RepairAgent

Prompt Section	Nature
Role	Static
Goals	Static
Guidelines	Static
State description	Dynamic
Available tools	Dynamic
Gathered information	Dynamic
Specification of output format	Static
Last executed command and result	Dynamic

(1) **Role**: provides the agent's area of expertise and outlines the agent's primary objective: finding and fixing bugs

(2) **Goals**: There are 5 goals for the agent to pursue:

- a) *Locate the bug* using testing and fault localization techniques

- b) *Gather information about the bug* by analyzing the relevant portion(s) of the codebase
- c) *Suggest simple fixes to the bug*
- d) *Suggest complex fixes*
- e) *Iterate over the previous goals*

(3) Guidelines: The LLM is provided with a set of guidelines. It is told that there are various kinds of bugs ranging from one-line-fix bugs to multi-line bugs. A list of recurring fix patterns is also provided that help solve many common bugs [Karampatsis and Sutton 2020] which is based on single-statement bugs in **Java**. The model is also instructed to add comments along with edited code which helps the model explain its reasoning and improve itself [Wei et al. 2022]. The model is also instructed to define a next step that can be converted into a call to a tool.

(4) State description: To guide the LLM toward using the tools effectively, a state machine is defined that constrains which tool(s) can be used at any given time. This is to stop the LLM to get lost in aimless exploration, which has happened very frequently as per the results of earlier tests. Fig. 2 shows the finite state machine. The set of tools are described in Section 3.4. The agent is free to move between any state via tools.

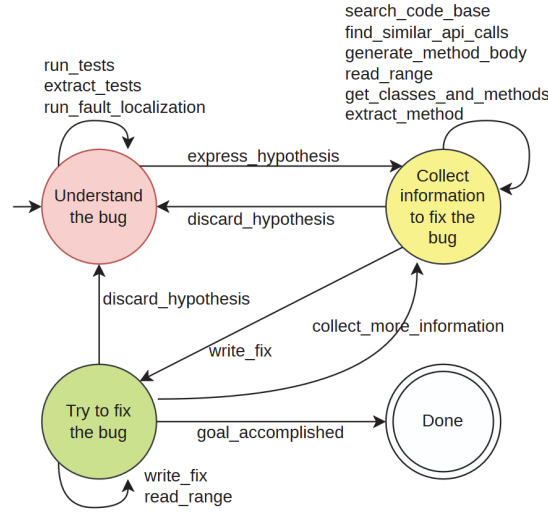


Fig. 2. State machine to guide selection of tools.

- *Understand the bugs:* The agent can collect information and bug location from testing results. It can then formulate a hypothesis based on its knowledge.
- *Collect information to fix the bug:* The agent can search for specific repair methods or read relevant code. After gathering enough informaton, it moves on to the next state.
- *Try to fix the bug:* The agent can try to fix the bug based on the hypothesis and information gathereed. If necessary, the agent can go back and establish a new hypothesis or gather additional information.

(5) Available tools:

(6) Gathered information:

(7) Specification of output format:

(8) Last executed command and result:

4 Findings/Results

Evaluations performed across the Defects4J benchmark indicate that RepairAgent effectively resolved 164 functional bugs, establishing capabilities beyond baseline tools like ChatRepair and ITER [???]. This performance is especially visible in multi-line fixes due to the contextual integration of search features [?]. Financial evaluations tracking token tracking usage configurations show an average financial overhead of approximately 14 cents per bug resolution attempt under standard model frameworks [?]. Further cross-validation against the GitBug-Java set shows functional adaptation across varied setups [?].

5 Discussion

Qualitative tracing indicates that access to structural tools yields better insights into functional anomalies [?]. However, complex modifications sometimes introduce unintended changes to straightforward single-line defects. Threats to internal and external validity include data leakage concerns during foundational model pre-training steps and reliance on highly precise localized metrics generated via secondary spectrum coverage frameworks like GZoltar [?].

References

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to fix bugs automatically. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 159:1–159:27. doi:10.1145/3360585
- Marcel Böhm, Ezekiel O. Soremekun, Sudipta Chattopadhyay, Ezekiel Ugherughe, and Andreas Zeller. 2017. Where is the bug and how is it fixed? an experiment with practitioners. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 117–128.
- Sébastien Bubeck, Varun Ext Chandrasekaran, Ronen Ext Eldan, Johannes Gehrke, Eric Horvitz, Ece Ext Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Ext Li, Scott M. Lundberg, Harsha Ext Nori, Hamid Ext Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. *CoRR abs/2303.12712* (2023). doi:10.48550/arXiv.2303.12712
- Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish K. Shevade. 2017. Deepfix: Fixing common C language errors by deep learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 1345–1351. <http://aaai.org/ocs/index.php/AAAI/AAAI17/paper/view/14603>
- Nan Ext Jiang, Kui Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 1430–1442. doi:10.1109/ICSE48619.2023.00125
- René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. 437–440.
- Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2023. Explainable automated debugging via large language model-driven scientific debugging. *CoRR abs/2304.02195* (2023). doi:10.48550/arXiv.2304.02195
- Rahul-Marie Karampatsis and Charles Sutton. 2020. How often do single-statement bugs occur? (Jun. 2020). doi:10.1145/3379597.3387491
- Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 802–811.
- Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987.
- Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. Genprog: A generic method for automatic software repair. *IEEE Trans. Software Eng.* 38, 1 (2012), 54–72.
- Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65. doi:10.1145/3318162
- Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. Tbar: revisiting template-based automated program repair. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 31–42. doi:10.1145/3293882.3330577
- Fan Long and Martin Rinard. 2016. Automatic patch generation by learning correct code. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312.
- Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781.
- Shishir G. Patil, Tianjun Ext Zhang, Xin Ext Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *CoRR abs/2305.15334* (2023). doi:10.48550/arXiv.2305.15334

- 261 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Ext Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023.
262 Toolformer: Language models can teach themselves to use tools. *CoRR* abs/2302.04761 (2023). doi:10.48550/arXiv.2302.04761
- 263 Michele Tufano, Juri Ext Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural
264 machine translation. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 25–36. <https://dl.acm.org/citation.cfm?id=3339509>
- 265 Jason Wei, Xuezhi Ext Wang, Dale Schuurmans, Maarten Bosma, Fei Ext Xia, Ed Ext Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting
266 elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, Vol. 35. 24824–24837.
- 267 Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023*
268 *IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1482–1494.
- 269 Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint*
(2023).
- 270 Jifeng Xuan, Matias Martinez, Favio Demarco, Maxime Clement, Sebastian Lamelas Marcote, Thomas Durieux, Daniel Le Berre, and Martin Monperrus.
271 2016. Nopol: Automatic repair of conditional statement bugs in java programs. *IEEE Transactions on Software Engineering* 43, 1 (2016), 34–55.
- 272 He Ye and Martin Monperrus. 2024. Iter: Iterative neural repair for multi-location patches. In *Proceedings of the International Conference on Software*
273 *Engineering (ICSE)*.